

Lab 05 - Bounded tool agent

AI Agents Fundamentals for Beginners | C690 | v1.0

Objective

Run a model-directed tool loop with schema checks and a strict read-only boundary.

Alignment: LO2. Scheduled hands-on time: 40 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- orders.csv
- policies.csv
- tool-contracts.json

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Open tool-contracts.json and orders.csv. Note that only lookup_order and search_policy are allowed, and the assigned customer is C-01.
2. Run python3 run.py to exercise the local tool boundary. Confirm cross-customer access and unknown tool names are denied.
3. Use PROMPTS.md prompt 1 in an approved LLM interface. Ask the model for one JSON tool proposal for order O-104, not a claimed result.
4. Save the proposed JSON as proposal.json and run python3 tools.py proposal.json. Copy the returned tool-result.json back into the same model conversation.
5. Allow at most six proposal/result rounds. Ask the model to produce a final cited draft when evidence is sufficient. Preserve the full observable tool trace in your evidence notes.
6. Test lookup_order for O-999 (wrong customer), issue_refund (not allowed) and an extra argument. Confirm actual code rejects each request, regardless of what the model says.
7. Optional: use agent.py with an approved OpenAI-compatible endpoint and your own environment variables to automate the same loop. The standard human-relay route requires no API key.

Acceptance checks

- Allowed lookup returns O-104 with a source record.
- Cross-customer lookup and write tools are rejected.
- The final answer relies on actual tool results and stops within six rounds.

Run python verify.py after run.py (use python3 if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's requirements.txt.

- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.
- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after the lab; keep source data and prompts unchanged.